

Návrh a implementace databázových systémů

Pokročilejší rysy jazyka SQL,
transakce, indexy

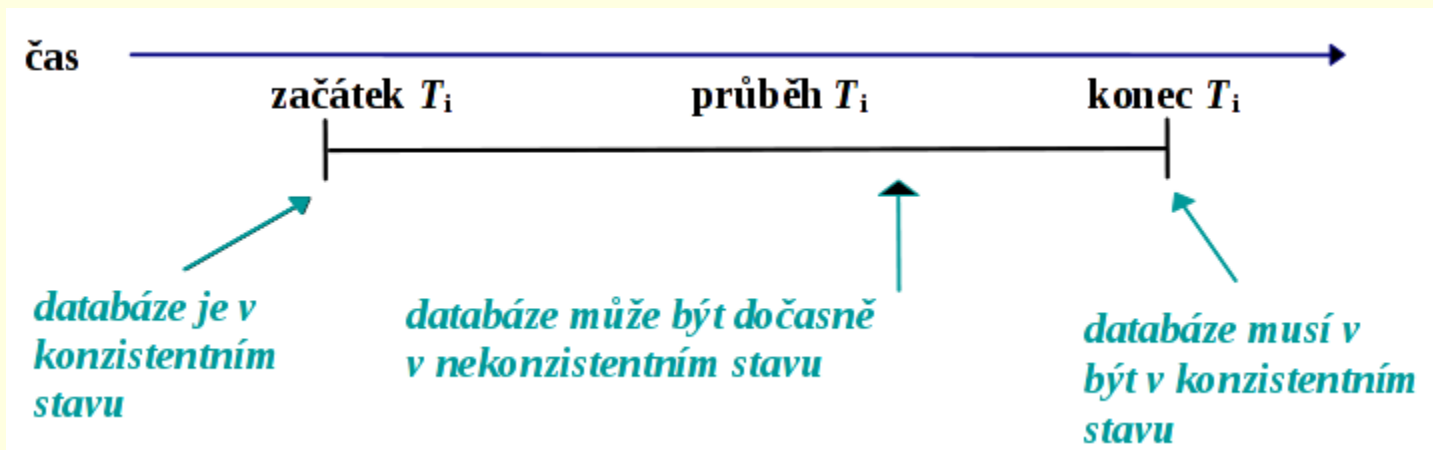
Transakce

- dva přirozené požadavky na databázi
 - zabezpečit data před chybami, haváriemi serveru apod.
 - stav (konzistence) DB není narušen chybou softwaru, systému, či média během operací, které mění data v DB
 - poskytnout korektní, rychlý a asynchronní přístup většímu množství současně pracujících uživatelů
 - každý musí vidět konzistentní stav DB bez ohledu na současný (concurrent) přístup ke stejným datům

Transakce

■ řešení – **transakce**

- mechanismus, který zajistí, že po skončení akce (korektním i nekorektním) bude DB konzistentní



- DBMS musí zaručit transparentní provádění transakcí

Transakce

- co to je transakce

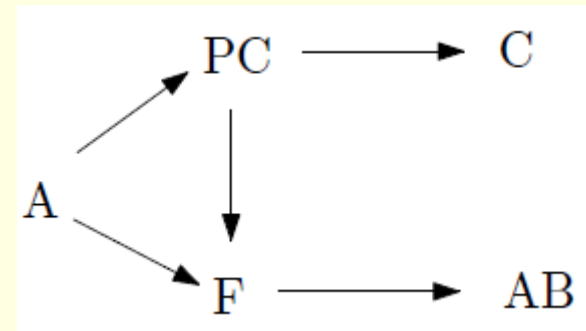
- atomická jednotka práce s databází, která převádí DB z jednoho konzistentního stavu do druhého
- je provedena buď celá, nebo vůbec
 - dokud není potvrzena, dílčí změny nejsou viditelné pro ostatní transakce
 - efekty úspěšné transakce jsou již trvalé
- skládá se obvykle z mnoha dílčích operací
 - PostgreSQL a jiné DBMS nicméně „obalují“ do samostatné transakce každý SQL příkaz

- příklady

- převod peněz z účtu na účet
- přihlášení na zkouškový termín

Transakce

- začátek a konec transakce
 - implicitní – řídí si DBMS
 - explicitní – příkazy **BEGIN**, **COMMIT**, **ROLLBACK**
- stavový diagram transakce



- Active – na začátku, probíhají DML příkazy
- Partially Committed – po poslední operaci
- Committed – potvrzený (po COMMITu)
- Failed – chyba, nelze pokračovat
- ABorted – po ROLLBACKu

Transakce

- vlastnosti transakcí ACID
 - atomicity – transakce proběhne celá nebo vůbec
 - consistency – transakce převádí DB z jednoho konzistentního stavu do jiného konzistentního stavu
 - independence – dílčí efekty jedné transakce nejsou viditelné ostatním transakcím
 - durability – efekty potvrzené transakce jsou trvalé a uložené

Zotavení z chyby

- globální chyby

- mají vliv na více transakcí

- pád serveru (ztráta vyrovnávací paměti)
 - uváznutí, ztráta komunikace – ovlivňují transakce, ale nikoli vlastní databázi
 - chyba média – ohrožení databáze či její části

- lokální chyby

- v jedné transakci

- chyby, které by měly být odchyceny
 - např. dělení nulou, porušení IO apod.
 - vyvolání ROLLBACKu

Zotavení z chyby po restartu

■ synchronizační bod

- časové známky (v žurnálu a v hlavičkách DB souborů), slouží k nalezení místa (v žurnálu) odkud je třeba začít s rekonstrukcí databáze

■ požadavky

- na transakce, jejichž stav bude v důsledku incidentu nedefinovaný, je nutné použít ROLLBACK
- potvrzené transakce, které nebyly dokončeny fyzicky (např. odeslání cache na disk), je nutné zopakovat a uložit do datových souborů

■ technicky dvě fáze

- Roll Forward – přehrání žurnálu (a obnova cache)
- Roll Back – odvolání transakcí, které nebyly v době pádu dokončeny

Problémy s paralelním zpracováním

- transakce není efektivní provádět za sebou
 - prokládání
 - vytváření tohoto rozvrhu zajišťuje rozvrhovač
- je nutno dávat pozor na konfliktní operace
 - READ a WRITE
- ošetření konfliktů
 - tzv. stupeň izolace transakce
 - čím vyšší, tím lepší je ochrana proti konfliktu, tím menší je ale prostupnost DB pro víceuživatelské operace
 - vyšší stupeň izolace také spotřebovává nějaké systémové zdroje

Problémy s paralelním zpracováním

■ lost update (ztráta aktualizace)

T1	T2	stav
READ(X) $X := X - 5$		$X = 80$ zrušení 5 rezervací
	READ(X) $X := X + 4$	$X = 80$ přidání 4 rezervací
WRITE(X)		$X = 75$
	WRITE(X)	$X = 84$

■ hodnota X je 84, ačkoli má být 79

Problémy s paralelním zpracováním

- dirty read (dočasná aktualizace)

T1	T2
READ(x) X:=X-5 WRITE(X)	
	READ(X) X:=X+4 WRITE(X)
READ(Z) -chyba transakce	

- po ROLLBACKu chybné transakce T1 bude aktualizace provedená transakcí T2 založena na chybných (později odvolaných) datech

Problémy s paralelním zpracováním

- non-repeatable read (neopakovatelné čtení)
 - T1 přečte hodnotu, T2 tuto hodnotu změní => další čtení této hodnoty transakcí T1 vrátí jiný výsledek než poprvé
- phantom read (fantom)
 - speciální případ neopakovatelného čtení
 - T1 čte data z tabulky (např. SELECT), mezitím do této tabulky T2 vloží nové řádky; T1 při dalším provedení téhož dotazu vrátí jiná data

Problémy s paralelním zpracováním

■ stupně izolace

	Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read
0	Read uncommitted	Possible	Possible	Possible
1	Read committed	Not possible	Possible	Possible
2	Repeatable read	Not possible	Not possible	Possible
3	Serializable	Not possible	Not possible	Not possible

■ v PostgreSQL

- lze nařídit stupně izolace 1 – 3 (při výběru read uncommitted se nastaví read committed)
 - při vyšším stupni izolace musí být aplikace připravena spouštět znovu transakce, jež se nepovede provést
- navíc v stupni 2 už není možný fantom
- příkaz SET TRANSACTION

Problémy s paralelním zpracováním

- v PostgreSQL

- příkaz `SET TRANSACTION parametr`

- kde *parametr* je mimo jiné

- `ISOLATION LEVEL {SERIALIZABLE | REPEATABLE READ | READ COMMITTED | READ UNCOMMITTED}`
 - příkaz se píše dovnitř transakce, před prvním příkazem transakce
 - výchozí nastavení je `READ COMMITTED`

Explicitní zamykání

- v PostgreSQL
 - pro různé příkazy stroj automaticky zamyká tabulky nebo řádky
 - zámek lze ale pořídit i explicitně
 - zámek je vždy držen až do konce aktuální transakce
- zámky na celých tabulkách (table-level)
 - příkaz `LOCK TABLE tabulka IN lockmode MODE`
 - kde *lockmode* je režim zámku (viz dokumentace)
 - různé režimy jsou různě tvrdé
 - omezují různé druhy přístupů ostatních transakcí k zamčené tabulce

Explicitní zamykání

- zámky na řádcích (row-level)
 - neblokují čtení, pouze zápis
- exclusive
 - automaticky zřízen např. při UPDATE nebo DELETE
 - získat ho lze tím, že se vybere daný řádek příkazem **SELECT FOR UPDATE**
- shared
 - získán příkazem **SELECT FOR SHARE**
 - víc transakcí může požádat o tento zámek, ale nikdo nemůže provést UPDATE, DELETE nebo požádat o exclusive lock – musí počkat, až transakce s shared zámkem skončí

Indexy

- odezvu DB lze zrychlit pomocí přídavných datových struktur, které usnadní vyhledávání
- pozor, každá přídavná struktura zvyšuje zároveň zátěž SŘBD, proto se musí užívat s rozmyslem
- index
 - přísluší tabulce (pozn.: pohled nelze indexovat)
 - informace, kde leží která data
 - využitelné pro SELECT, UPDATE, DELETE, JOIN
- je to podobné, jako rejstřík knihy
 - najdeme si v rejstříku, kde leží požadovaná informace, a nalistujeme příslušnou stránku

Indexy

- něco o fyzickém uspořádání souborů na disku
 - hromada
 - neuspořádaný soubor záznamů proměnné délky
 - nalezení záznamu má složitost $O(N)$
 - sekvenční soubor
 - neuspořádaný resp. uspořádaný soubor záznamů stejné délky
 - nalezení záznamu má složitost $O(N)$ resp. $O(\log N)$
 - indexsekvenční soubor
 - sekvenční soubor setříděný dle PK plus přídatný víceúrovňový index
 - index neukazuje na jednotlivé záznamy, ale na krátké setříděné bloky více záznamů

Indexy

- něco o fyzickém uspořádání souborů na disku
 - indexovaný soubor
 - sekvenční soubor (nemusí být setříděný ani souvislý)
plus přídatný jednoúrovňový index
 - index ukazuje na jednotlivé záznamy
 - víceúrovňový index
 - indexace indexu
 - řekněme, že indexujeme podle sloupce (celá čísla),
pak první úroveň indexu ukazuje na jednotlivá čísla,
druhá indexuje první číslo z každé pětice indexů první
úrovně (třeba), atd...

Indexy

- v praxi je fyzické uspořádání souborů v principu dvojí, setkáme se s označením
 - index-organized
 - záznamy jsou umístěny přímo ve struktuře indexu
 - heap-organized
 - non-clustered – záznamy nejsou setříděné
 - clustered – záznamy jsou setříděny dle určitého indexu

Indexy

- možnosti jsou opět závislé na použitém DBMS
 - Oracle umožňuje dva typy fyzického uložení tabulek
 - index-organized – záznamy jsou uloženy do struktury podobné B-stromu, tj. přímo do indexu
 - heap-organized – záznamy uloženy za sebou, bez řazení
 - navíc lze společně ukládat propojené tbl do *clusteru*
 - společné sloupce uloženy pouze jednou
 - PostgreSQL má tabulky vždy heap-organized, ale umí je fyzicky řadit dle zadaného indexu
 - příkaz **CLUSTER**

pojem CLUSTER je používán v mnoha různých významech!

Indexy

- vyhledávání v indexu
 - lze zjevně použít binární vyhledávací strom
 - každý uzel X má dva potomky, levý potomek je vždy menší než uzel X , pravý potomek je vždy větší než X
 - složitost prohledávání $O(\log_2 N)$ – to může být pro velká množství záznamů stále ještě příliš mnoho (uvažme opakované přístupy na disk...)
 - B-stromy
 - každý uzel má m potomků
 - snížení hloubky stromu $\Rightarrow$ omezení počtu přístupů na disk
 - obvyklý počet úrovní stromu 2-3 pro stovky záznamů

Indexy

■ příklad

- chceme vyhledat jeden záznam podle jeho PK
- pokud o uspořádání záznamů v tabulce nic nevíme, musíme projít celou tabulku
- pokud máme index, lze hledání výrazně urychlit
 - vytvoříme index nad sloupcem, podle kterého často vyhledáváme
 - lze tvořit index pro více sloupců naráz

```
CREATE TABLE test ( major integer, minor integer, content varchar );  
CREATE INDEX test_mm_index ON test (major, minor);  
SELECT content FROM test WHERE major = 5 AND minor = 10;  
DROP INDEX test_mm_index;
```

Indexy

■ PostgreSQL

- umí tvořit indexy různými technikami
 - B-strom je implicitní
- umí analyzovat, zda je index užíván či nikoli
- umí indexovat skalární funkce dat
 - lze tak vynutit dodržení některých IO, např. to, že se ve sloupci nesmí opakovat stejný řetězec s malými či velkými písmeny

```
CREATE UNIQUE INDEX test_lower_col1_idx ON test (lower(col1));
```

- umí vytvářet index jen na podmnožinu záznamů

```
CREATE INDEX orders_unbilled_idx ON orders (order_nr)  
WHERE billed is not true;
```